نکات عملگرهای RxJS

جمع‌آوری فایل‌های txt داخل فولدرهای src/app. متن هر بخش همان یادداشت اصلی است.

عملگرهای ساخت (Creation)

- of
- from
- interval
- timer

عملگرهای ترکیب (Combination)

- merge
- concat
- zip
- combineLatest
- forkJoin
- race
- withLatestFrom
- startWith

عملگرهای تبدیل (Transformation)

- buffer
- bufferCount
- bufferTime
- bufferToggle
- bufferWhen
- concatMap
- mergeMap
- exhaustMap
- scan
- pairwise

عملگرهای فیلتر (Filtering)

- debounce
- debounceTime
- throttle
- throttleTime
- distinct
- first
- last
- sample
- skip
- skipUntil
- skipWhile
- take
- takeLast
- takeUntil
- takeWhile

عملگرهای شرطی و کمکی

- defaultIfEmpty
- every
- findIndex
- sequenceEqual
- repeat

عملگرهای ساخت (Creation)

of

منبع: of/of.txt

یک Creation Operator بسیار پرکاربرد است که تعدادی مقدار (متغیر یا ثابت) را می‌گیرد، به ترتیب آن‌ها را منتشر می‌کند و بلافاصله استریم را complete می‌کند.

در کد تصویر: of(1) عدد 1 را منتشر کرده و بلافاصله استریم را می‌بندد.
می‌توانید چندین مقدار هم به آن بدهید (مثلاً of(10, 20, 30) که هر سه را پشت سر هم منتشر کرده و تمام می‌شود).

همگام (Synchronous): مقادیر را به صورت آنی و بدون هیچ تأخیری پشت سر هم منتشر می‌کند.
شباهت با from: برخلاف from که یک آرایه یا پرامیس را می‌شکافد و اقلامش را تک‌تک بیرون می‌کشد، اپراتور of هر ورودی را دقیقاً همان‌طور که هست (حتی اگر خودش یک آرایه باشد) به عنوان یک آیتم واحد منتشر می‌کند.

from

منبع: from/from.txt

تبدیل داده‌های معمولی به استریم (from): هر آرایه، پرامیس یا شیء قابل پیمایشی را به راحتی به یک Observable تبدیل می‌کند تا قابلیت پایت کردن (pipe) پیدا کند.

interval

منبع: interval/interval.txt

یک Creation Operator است که اعداد صحیح را به صورت دنباله‌دار و با فاصله‌های زمانی مشخص (بر حسب میلی‌ثانیه) منتشر می‌کند (دقیقاً مثل تابع setInterval در جاوااسکریپت).

در کد تصویر: interval(10) یعنی هر ۱۰ میلی‌ثانیه یک بار، یک عدد جدید منتشر کن.
مقادیر انتشاری: همیشه از عدد 0 شروع می‌شود و به ترتیب افزایشی ادامه پیدا می‌کند (0, 1, 2, 3, ...).

بی‌پایان (Infinite): به طور پیش‌فرض هیچ‌وقت تمام نمی‌شود complete) نمی‌شود(و تا ابد ادامه دارد، مگر اینکه خودتان با اپراتورهایی مثل take یا takeUntil متوقفش کنید.

کاربرد رایج: ساخت تایمر، پولینگ مداوم از سرور (Polling) برای دریافت داده‌های جدید، یا ایجاد تاخیرهای چرخه‌ای در انیمیشن‌ها و استریم‌ها.

timer

منبع: timer/timer.txt

تابع timer یک Creation Operator است که زمان‌بندی انتشار داده را مدیریت می‌کند. این تابع دو حالت اصلی دارد:

دو آرگومانی ( )timer(dueTime, period مثل تصویر(:

آرگومان اول (30): تاخیر اولیه (Initial Delay)؛ یعنی ابتدا ۳۰ میلی‌ثانیه صبر کن، بعد عدد 0 را منتشر کن.
آرگومان دوم (10): دوره تکرار (Interval)؛ بعد از شروع، هر ۱۰ میلی‌ثانیه یک‌بار عدد بعدی (1, 2, 3, ...) را منتشر کن.
تک آرگومانی (30)timer:

اگر آرگومان دوم را ندهید، فقط ۳۰ میلی‌ثانیه صبر کرده، مقدار 0 را منتشر می‌کند و بلافاصله complete می‌شود)جایگزین مدرن setTimeout در دنیای (Reactive.

تفاوت کلیدی با interval:

در interval(10)، انتشار اولین عدد هم ۱۰ میلی‌ثانیه بعد اتفاق می‌افتد و همیشه فاصله بین همه تیک‌ها ثابت است.

در timer(30, 10)، می‌توانید تعیین کنید شروع استریم با چه میزان تاخیر دلخواهی باشد و بعد از آن با چه سرعتی تکرار شود (یا حتی اصلاً تکرار نشود).

دقیقاً بله، فرمولش دقیقاً همین است:

حالت تک‌آرگومانی timer(dueTime):

مثل setTimeout عمل می‌کند.

به اندازه زمان مشخص‌شده صبر می‌کند، فقط یک عدد (0) منتشر می‌کند و بلافاصله Complete می‌شود.

حالت دوآرگومانی timer(dueTime, period):

مثل ترکیبی از setTimeout + setInterval عمل می‌کند.

بعد از تاخیر اولیه، شروع به تکرار نامحدود می‌کند (0, 1, 2, ...) و هیچ‌وقت به خودیِ خود Complete نمی‌شود؛ مگر اینکه خودتان آن را unsubscribe کنید یا با اپراتورهایی مثل take() یا takeUntil() ببندید.

عملگرهای ترکیب (Combination)

merge

منبع: merge/merge.txt

عملگر merge ساده‌ترین نوع ترکیب است: «چند جریان را به یک جریان تبدیل کن».

به همه Observableها همزمان Subscribe می‌شود و هر کدام مقداری فرستاد، همان لحظه بدون تغییر به خروجی می‌رود. هیچ pairing، صبر یا ترکیبی در کار نیست — فقط ادغام زمانی (Chronological).

.مقادیر 20, 40, 60, 80, 100 را با فاصله منظم می‌فرستد $x
.دو بار مقدار 1 را در لحظات مختلف می‌فرستد $y
در خروجی، مقادیر دقیقاً به ترتیب زمان وقوع ردیف شده‌اند:
20 → 40 → 60 → (1 از $y) → 80 → 100 → (1 از $y)

می‌بینید که 1 اول دقیقاً در فاصله بین 60 و 80 افتاده، چون در تایم‌لاین $y همان موقع منتشر شده بود.
merge هیچ دخالتی در مقادیر ندارد؛ فقط پخش‌کننده زمانی است.

نکته: خروجی merge وقتی Complete می‌شود که همه ورودی‌ها Complete شوند.

وقتی چند منبع رویداد دارید که باید یک رفتار مشترک داشته باشند. مثال کلاسیک: رفرش داده هم به‌صورت خودکار (تایمر) و هم دستی (کلیک دکمه):

```
merge(
  interval(30_000),
  this.refreshButtonClicks$
).pipe(
  switchMap(() => this.http.get<Stat[]>('/api/stats'))
).subscribe(stats => this.stats = stats);
```

⚠️ نکته: چون merge به همه همزمان گوش می‌دهد، اگر ورودی‌ها هرگز Complete نشوند (مثل کلیک‌ها و تایمرها)، خروجی هم هیچ‌وقت تمام نمی‌شود — در کامپوننت‌های Angular معمولاً مشکلی نیست (با destroy شدن subscription قطع می‌شود)، ولی در سرویس‌های app-level حواستان به نشتی حافظه باشد.

concat

منبع: concat/concat.txt

اول تمام مقادیر Observable اول را ارسال می‌کند و فقط زمانی که Observable اول complete شود (خط عمودی |)، به سراغ دومی می‌رود.

جریان $x:

مقادیر 1، 1 و 1 را در طول زمان ارسال می‌کند.
سپس علامت | آمده؛ یعنی $x بسته شد (Complete).
تا قبل از اینکه $x بسته شود، اصلاً کاری با $y ندارد (Subscribe روی $y انجام نمی‌شود).
جریان خروجی:

تمام 1های $x بدون وقفه ارسال می‌شوند.
به محض رسیدن به خط عمودی $x، حالا سراغ $y می‌رود و شروع به گوش دادن به آن می‌کند.

اگر Observable اول هیچ‌وقت Complete نشود (مثل گوش دادن به کلیک دکمه یا وب‌سوکت دائمی)، Observable دومی هرگز اجرا نخواهد شد!

درخواست‌های پشت سر هم (Waterfalling Requests):مثلاً اول ذخیره در دیتابیس/سرور، و دقیقاً بعد از اتمام موفقیت‌آمیز آن، فراخوانی رفرش لیست:

```
concat(saveData$, refreshList$).subscribe();
```

اجرای انیمیشن‌های پشت‌سرهم: اول انیمیشن خروج کارت، بعد لود شدن انیمیشن کارت جدید.

zip

منبع: zip/zip.txt

عملگر zip یعنی: «مقادیر را مثل زیپ (بسته‌بندی) یکی‌درمیان جفت کن — اولی با اولی، دومی با دومی».

امین مقدار» برسد، سپس آن‌ها را به‌صورت یک آرایه منتشر-n» منتظر می‌ماند تا از همه‌ی استریم‌ها zip می‌کند. یعنی جفت‌سازی بر اساس ترتیب (شماره) انجام می‌شود، نه زمان. مقداری که زودتر رسیده، در حافظه بافر می‌شود تا نوبت هم‌جفتش برسد.

تحلیل دیاگرام:

```
zip(x$, y$)
```

1 از $x می‌رسد → هنوز جفت ندارد، در حافظه می‌ماند. سپس A می‌رسد → جفت اول کامل شد → خروجی: 1A

2 می‌رسد → منتظر می‌ماند... B می‌رسد → خروجی: 2B

دقیقاً در C — به‌تنهایی خروجی‌ای نمی‌سازد!) → بعد 3 می‌رسد → خروجی: C 3 :می‌رسد (توجه emit لحظه‌ی دیرترین عضو جفت اتفاق می‌افتد.

هم به همین صورت 4D.

⚠️ مهم‌ترین نکته‌ی دیاگرام: 5 از $x می‌آید، اما $y فقط ۴ تا مقدار داشت و تمام شد! 5 هیچ‌وقت منتشر نمی‌شود — بدون شریکِ پنجم، تا ابد در بافر می‌ماند و دور انداخته می‌شود.

کمتر از بقیه استفاده می‌شود، ولی برای سناریوهای «تطبیق ترتیبی» عالی است UI در کد روزمره‌ی zip:

```
// فایل‌ها با شناسه‌های سرور، به ترتیب
zip(files$, serverIds$).pipe(
  map(([file, id]) => ({ ...file, id }))
).subscribe(paired => this.items = paired);
```

۲. Rate-limiting (حداکثر یک عملیات در ثانیه) دستی

```
zip(this.pendingRequests$, interval(1000)).pipe(
  map(([req]) => req)
).subscribe(req => this.http.send(req));
```

combineLatest

منبع: combine-latest/combineLatest.txt

هر وقت هر کدام از Observableها مقداری بفرستد، آخرین مقدارِ همه آن‌ها با هم ترکیب شده و به خروجی می‌رود.

قانون مهم: قدم اول دوطرفه است ⚠️
combineLatest حداقل یک مقدار نفرستاده باشند، هیچ خروجی‌ای Observable تا زمانی که هر دو نمی‌دهد.

مثل این است که دو ورودی فرم دارید — مثلاً نام کاربری ($x) و رمز عبور ($y) — و می‌خواهید هر بار هرکدام تغییر کرد، آخرین وضعیت هر دو را داشته باشید:

```
combineLatest([username$, password$]).pipe(
  map(([user, pass]) => ({ user, pass }))
).subscribe(form => validateForm(form));
```

forkJoin

منبع: forkjoin/forkjoin.txt

تا زمانی که تمام Observableها کامل نشوند (Complete)، هیچ خروجی‌ای داده نمی‌شود. وقتی همه کامل شدند، فقط «آخرین مقدار» هرکدام را برمی‌دارد و ترکیب می‌کند.

صبر طولانی: forkJoin به محض اجرا، به $x و $y گوش می‌دهد، اما هیچ‌چیزی به خروجی نمی‌فرستد (برخلاف combineLatest که با هر تغییر واکنش نشان می‌داد).
شرط تکمیل: منتظر می‌ماند تا $x تمام شود (رسیدن به |) و $y هم تمام شود (رسیدن به |).
ترکیب نهایی: فقط در لحظه آخر، آخرین مقداری که $x فرستاده بود (C) و آخرین مقداری که $y فرستاده بود (2) را برمی‌دارد، با هم ترکیب می‌کند (C2) و سپس خودش هم Complete می‌شود.

فرض کنید می‌خواهید پروفایل کاربر را بگیرید و لیست تنظیماتش را هم همزمان از دو API مختلف دریافت کنید:

```
forkJoin({
  profile: this.http.get('/api/user'),
  settings: this.http.get('/api/settings')
}).subscribe(({ profile, settings }) => {
  // موفقیت‌آمیز تمام شوند HTTP این کد فقط زمانی اجرا می‌شود که هر دو درخواست
```

```
  this.initPage(profile, settings);
});
```

⚠️ هشدار: اگر یکی از Observableها (مثلاً یکی از APIها) خطا بدهد یا اصلاً Complete نشود، forkJoin هیچ خروجی‌ای نخواهد داد. این یکی از شایع‌ترین باگ‌ها در استفاده از این عملگر است.

race

منبع: race/race.txt

عملگر race (دقیقاً معادل مسابقه دو یا رقابت) است. معادل آن در پرامیس‌های جاوااسکریپت Promise.race است.

به تمام Observableها همزمان گوش می‌دهد؛ هر کدام که «اولین مقدار» را زودتر از بقیه بفرستد، برنده مسابقه است! بقیه بازنده‌ها بلافاصله (Unsubscribeدور انداخته(می‌شوند و از آن لحظه به بعد، خروجی فقط و فقط مقادیر برنده را منعکس می‌کند.

شروع مسابقه: هر سه جریان $x و $y و $z در یک لحظه شروع می‌شوند.
برنده مشخص می‌شود:
اولین مقداری که در کل تایم‌لاین شلیک می‌شود، عدد 1 از استریم $y است.
به محض انتشار 1، مسابقه تمام می‌شود: $y برنده شد!
سرنوشت بازنده‌ها:
استریم‌های $x و $z بلافاصله لغو (Unsubscribe) می‌شوند. مقادیر 20, 40, 60 از $x و 0, 0, 0 از $z کلاً نادیده گرفته می‌شوند.
خروجی نهایی:
خروجی فقط مقادیر $y را تا انتها دنبال می‌کند: 1 -> 2 -> 3.

رقابت بین پاسخ سرور و یک تایمر:

```
race(
  this.http.get('/api/heavy-data'),
  timer(5000).pipe(map(() => { throw new Error('Timeout: سرور پاسخ نداد!'); }))
).subscribe({
  next: data => console.log('داده دریافت شد', data),
  error: err => console.error(err)
});
```

اگر سرور زیر ۵ ثانیه پاسخ دهد، برنده است. اگر بیش از ۵ ثانیه طول بکشد، تایمر برنده شده و ارور پرتاب می‌شود و ریکوئست لغو می‌شود.

۲. واکشی داده از Cache سریع‌تر یا Network:

```
race(
  this.fetchFromIndexedDB(),
  this.fetchFromNetwork()
)
```

هر کدام زودتر جواب داد استفاده می‌شود و دیگری لغو می‌گردد.

withLatestFrom

منبع: with-latest-from/with-latest-from.txt

عملگر withLatestFrom یعنی: «هر وقت من (استریم اصلی) مقدار فرستادم، آخرین مقدارِ آن استریم دیگر را هم بردار و با هم بده».

استریم اصلی ($x) فرمانده است؛ خروجی فقط و فقط وقتی تولید می‌شود که $x منتشر کند. در آن لحظه، آخرین مقدار موجود از $y برداشته و جفت می‌شود. انتشارِ $y به‌تنهایی هیچ خروجی‌ای نمی‌سازد.

```
x$.pipe(
  withLatestFrom(y$),
  map((x, y) => x + y)
)
```

هنوز هیچ چیزی نفرستاده! نتیجه: این مقدار گم می‌شود و خروجی $y مقدار 1 را می‌فرستد → اما $x نمی‌دهد. (مهم‌ترین نکته‌ی دیاگرام)

Aاست → خروجی: A 2 تا این لحظه $y مقدار 2 → آخرین مقدار $x

به‌تنهایی Bو Cو B :توجه) Dاست → خروجی: D 3 فرستاده؛ آخرینش A, B, C, D تا الان $y → مقدار 3 $x

x** هیچ خروجی‌ای نساختند

سناریوی کلاسیک: «کلیک کاربر + آخرین وضعیت فرم/استور». کلیک تایمینگ را تعیین می‌کند، ولی به آخرین state نیاز دارید:

```
saveClicks$.pipe(
  withLatestFrom(this.formValue$),        // (click, formValue)
  switchMap(([, form]) => this.api.save(form))
).subscribe();
```

startWith

منبع: start-with/startWith.txt

عملگر startWith ساده‌ترین عضو این خانواده است: «اول این مقدار را بفرست، بعد برو سراغ جریان اصلی».

قبل از اینکه Observable اصلی حتی یک مقدار بفرستد، مقدار(های) داده‌شده به startWith را فوراً منتشر می‌کند. یعنی خروجی از همان لحظه صفر Subscribe) شدن(مقدار دارد.

تحلیل دیاگرام:

```
x$.pipe(startWith(1))
```

جریان اصلی x$ مقادیر 2 و 3 را کمی دیرتر می‌فرستد (می‌بینید که در ابتدای تایم‌لاین فاصله خالی است).
در خروجی، مقدار 1 بلافاصله در لحظه صفر ظاهر می‌شود (سبز بعد از آن از همه).
بعد از آن، جریان اصلی دست‌نخورده ادامه پیدا می‌کند: 2 و 3.
نکته: می‌توانید چند مقدار بدهید: startWith('a', 'b') — به ترتیب پشت سر هم منتشر می‌شوند.

بدون startWith، تا کاربر کلیک نکند صفحه خالی می‌ماند.

مثال ۲: ترکیب با) combineLatestحل مشکل «قدم اول دوطرفه»(
یادتان هست گفتم combineLatest تا هر دو استریم مقدار نفرستند خروجی نمی‌دهد؟ راه‌حل استاندارد همین است:

```
combineLatest([
  firstName$.pipe(startWith('')),
  lastName$.pipe(startWith(''))
]).pipe(
  map(([f, l]) => `${f} ${l}`.trim())
).subscribe(fullName => this.fullName = fullName);
```

حالا از لحظه صفر خروجی دارید (رشته خالی)، و با اولین تایپ هم فرم زنده آپدیت می‌شود.

عملگرهای تبدیل (Transformation)

buffer

منبع: buffer/buffer.txt

عملگر buffer متعلق به دسته‌ی Transformation Operators (عملگرهای تبدیل) در RxJS است. هدف اصلی آن تبدیل یک جریان از مقادیر تک‌به‌تک (Item-by-Item) به یک جریان از آرایه‌ها (Batches) بر اساس یک شرط/سیگنال خارجی است.

```
x$.pipe(
  buffer(y$)
)
```

ایجاد یک آرایه داخلی (Internal Buffer): RxJS یک آرایه خالی در حافظه می‌سازد: [].

گوش دادن به x$: هر مقداری که از منبع اصلی بیاید (A, B, ...)، بدون این‌که مستقیم به خروجی پاس داده شود، درون این آرایه push می‌شود.

گوش دادن به (Closing Notifier) y$:

به محض اینکه y$ یک مقدار (هر چیزی، حتی 0، null یا event) منتشر کند:

کپی آرایه فعلی به عنوان یک مقدار خروجی Emit می‌شود.

آرایه داخلی کاملاً ریست (خالی) می‌شود ([] = []).

چرخه بعدی: تمام آیتم‌های جدید تا انتشار بعدی از سمت y$ در آرایه جدید جمع می‌شوند.

مثال‌های واقعی و عملی

می‌خواهید تشخیص دهید کاربر چند بار در یک بازه زمانی کوتاه (مثلاً ۴۰۰ میلی‌ثانیه) روی دکمه کلیک کرده است:

```
import { fromEvent, debounceTime, buffer, map, filter } from 'rxjs';

const click$ = fromEvent(buttonElement, 'click');

const multiClicks$ = click$.pipe(
  // y$: بعد از ۴۰۰ میلی‌ثانیه سکوت، یک پالس می‌فرستد
  buffer(click$.pipe(debounceTime(400))),
  // تعداد کلیک‌های جمع‌شده در این بازه
  map(clicks => clicks.length),
  // فیلتر کردن مواردی که ۲ بار یا بیشتر کلیک شده
  filter(count => count >= 2)
);

multiClicks$.subscribe(count => {
  console.log(`کاربر ${count} بار کلیک کرد!`);
});
```

```
});
```

bufferCount

منبع: buffer-count/buffer-count.txt

عملگر bufferCount مقادیر را بر اساس تعداد مشخص دسته‌بندی و به شکل آرایه منتشر می‌کند.

در تصویر از حالت دو آرگومانی آن استفاده شده:

```
bufferCount(bufferSize, startBufferEvery)

 bufferCount(3, 2)
```

گام‌به‌گام دیاگرام:

مقدار A: بافر اول باز می‌شود

→

→

[A]

مقدار B: به بافر اول اضافه می‌شود

→

→

[A, B]

مقدار C:

چون ۲ آیتم رد شد، بافر دوم باز می‌شود با [C].

بافر اول به ۳ آیتم رسید

→

→

خروجی: [A, B, C]

مقدار D: به بافر دوم اضافه می‌شود

→

→

[C, D]

مقدار E:

مجدداً ۲ آیتم رد شد، بافر سوم با [E] باز می‌شود.

بافر دوم به ۳ آیتم رسید

→

→

خروجی: [C, D, E]

مقدار F: به بافر سوم اضافه می‌شود

→

→

(می‌شود emit در صورت بسته‌شدن استریم، مانده بافر) [E, F]

bufferTime

منبع: buffer-time/buffer-time.txt

در تصویر: bufferTime(30) یعنی هر ۳۰ واحد زمانی (معمولاً میلی‌ثانیه)، یک «پنجره» بسته شده و هر چه تا آن لحظه جمع شده، به شکل آرایه منتشر می‌شود.

گام‌به‌گام دیاگرام:

در بازه اول، مقادیر A، B و C می‌آیند

→

→

در پایان بازه: خروجی [A, B, C]

در بازه دوم، هیچ مقداری نمی‌آید

→

→

خروجی آرایه خالی [] است (نقطه نارنجی در تصویر).

در بازه سوم، D و E می‌آیند

→

→

خروجی [D, E]

.بعد از این بافر می‌آید و در پنجره بعدی منتشر می‌شود F

ارسال دسته‌ای (Batch) به سرور: مثلاً جمع‌کردن لاگ‌ها یا رویدادهای کاربر و ارسال یک‌جا هر ۵ ثانیه:

```
logs$.pipe(
  bufferTime(5000),
  filter(batch => batch.length > 0) // جلوگیری از ارسال آرایه خالی
).subscribe(batch => api.send(batch));
```

bufferToggle

منبع: buffer-toggle/buffer-toggle.txt

```
bufferToggle(
  openings$,                    // y$: چه زمانی یک بافر «باز» شود
  closingSelector: (openValue) => Observable  // هر بافر «کی» بسته شود
)
```

یعنی: هر بار $y مقدار داد (مثلاً 10)، یک بافر باز می‌شود که بعد از 10 واحد زمانی (توسط timer(10)) بسته می‌شود.

مقادیر 1 و 2 از $x می‌آیند

→

→

هیچ بافری وجود ندارد، پس دور ریخته می‌شوند.

شروع بافر اول ($y مقدار 10 می‌دهد):

یک بافر جدید باز می‌شود.

شرط بسته‌شدن: timer(10) یعنی بافر فقط به مدت ۱۰ واحد زمانی باز می‌ماند.

در طول این بازه کوتاه، فقط مقدار 3 می‌آید و ذخیره می‌شود.

تایمر ۱۰ تمام می‌شود

→

→

بافر بسته شده و امیت می‌شود: [3]

فاصله بین دو بافر (بافری باز نیست):

مقادیر 4 و 5 می‌آیند

→

→

چون بافری باز نیست، دور ریخته می‌شوند.

شروع بافر دوم ($y مقدار 30 می‌دهد):

یک بافر جدید باز می‌شود.

شرط بسته‌شدن: timer(30) یعنی بافر به مدت طولانی‌تر (۳۰ واحد زمانی) باز می‌ماند.

در طول این ۳۰ واحد زمانی، مقادیر 6، 7 و 8 می‌آیند و داخل بافر قرار می‌گیرند.

تایمر ۳۰ تمام می‌شود

→

→

بافر بسته شده و امیت می‌شود: [6, 7, 8]

پایان:

مقدار 9 بعد از بسته شدن بافر دوم می‌آید

→

→

چون بافری باز نیست، نادیده گرفته می‌شود.

bufferWhen

منبع: buffer-when/buffer-when.txt

این فایل خالی است.

concatMap

منبع: concat-map/concat-map.txt

Observable مقادیر ورودی را به صورت سریالی (صف) پردازش می‌کند؛ یعنی تا زمانی که concatMap نشود، پردازش مقدارِ جدید را شروع نمی‌کند Complete داخلیِ قبلی.

توقف کل زنجیره: اگر Observable داخلی)اینجا ($y هیچ‌وقت کامل نشود، پردازشِ مقادیر بعدیِ $x برای همیشه متوقف می‌شود.

رفتار "صف" انتظار: مقادیر ورودیِ $x حذف نمی‌شوند، بلکه پشت درِ "بسته" می‌مانند و چون نوبت به آن‌ها نمی‌رسد، هرگز پردازش نخواهند شد.

قربانی کردن عملکرد برای ترتیب: این عملگر به‌گونه‌ای طراحی شده که ترتیب خروجی‌ها را حفظ کند؛ بنابراین این "بلوک شدن" بهایِ تضمینِ ترتیبِ خروجی‌هاست.

نیاز به مدیریت دستی: در سناریوهایی که احتمالِ عدمِ تکمیلِ Observable داخلی وجود دارد، حتماً باید از take(n) (برای بستن اجباری) یا timeout (برای مدیریت خطا) استفاده کنید.

انتخاب جایگزین: اگر نیاز دارید پردازش‌ها بدونِ انتظار برای تکمیلِ قبلی انجام شوند (بدون بلوک شدن)، به جای concatMap از mergeMap استفاده کنید.

mergeMap

منبع: merge-map/merge-map.txt

مثل پیش‌خوانی است که بدون توجه به خریدار قبلی، سفارش هر مشتری جدید را همزمان به mergeMap آشپزخانه می‌فرستد و هر غذایی که زودتر پخته شد را فوراً تحویل می‌دهد.

پردازش کاملاً همزمان (Parallel): برای هر ورودی از استریم بیرونی ($x)، استریم درونی ($y) بلافاصله اجرا می‌شود.
بدون صف، بدون لغو: نه مثل concatMap ورودی بعدی را منتظر نگه می‌دارد، نه مثل switchMap قبلی را لغو (Cancel) می‌کند.
خروجی به ترتیب سرعت (Interleaving): تقدم خروجی‌ها فقط بر اساس «زمان آماده شدن داده» است؛ داده‌های استریم‌های مختلف می‌توانند درهم‌تنیده شوند.
طول عمر وابسته به Complete: هر استریم درونی به کارش ادامه می‌دهد تا تمام داده‌های خودش (1 و 2 و 3) را تحویل دهد و بسته شود.

exhaustMap

منبع: exhaust-map/exhaust-map.txt

درخواست جدید را پردازش می‌کند و تا زمانی که تمام نشده، تمام درخواست‌های بعدی را exhaustMap نادیده می‌گیرد.

scan

منبع: scan/scan.txt

دقیقاً معادل Array.prototype.reduce اما پیوسته: یک مقدار انباشته (Accumulator) را در طول زمان نگه می‌دارد.
انتشار لحظه‌به‌لحظه (Intermediate Results): برعکس reduce در RxJS که فقط در زمان complete شدن استریم یک خروجی نهایی می‌دهد، scan با هر ورودی جدید بلافاصله نتیجه تجمیعی را منتشر می‌کند:

مدیریت State در RxJS: ابزار پایه‌ای برای پیاده‌سازی الگوهای Redux/State Management بدون نیاز به کتابخانه جانبی)ترکیب اکشن‌های جدید با State قبلی(.

```
پشتیبانی از مقدار اولیه (Initial Seed): می‌توانید آرگومان دوم به آن بدهید (مثل scan((acc, curr) => ..., initialValue)).
```

pairwise

منبع: paire-wise/paire-wise.txt

جفت‌کردن داده‌ها: داده‌ی قبلی و داده‌ی فعلی را در قالب یک آرایه دو عضوی [prev, current] تحویل می‌دهد.
شروع از مقدار دوم: تا زمانی که حداقل ۲ مقدار منتشر نشود، هیچ خروجی نمی‌دهد (مقدار اول A به‌تنهایی چیزی تولید نمی‌کند).
پنجره‌ی لغزان (Sliding Window): با هر مقدار جدید، یک گام جلو می‌رود و مقدار قبلی را با جدید جفت می‌کند ([A, B]، سپس [B, C]، سپس [C, D] و...).
کاربرد اصلی در انگولار/فرانت: مقایسه وضعیت قبلی با فعلی (مثل محاسبه جهت اسکرول، تغییرات فیلترهای فرم، یا ردیابی Route قبلی و فعلی در Router.events).

.مقدار قبلی و فعلی استریم را دوبه‌دو به شکل یک جفت [قبلی, جدید] به شما تحویل می‌دهد pairwise

عملگرهای فیلتر (Filtering)

debounce

منبع: debounce/debounce.txt

قانون کلی debounce:

هرگاه مقداری ارسال شود، یک تایمر اختصاصی برایش فعال می‌شود. اگر قبل از اتمام آن تایمر مقدار جدیدی بیاید، مقدار قبلی دور ریخته شده و تایمر از اول برای مقدار جدید محاسبه و ست می‌شود

debounceTime

منبع: debounce-time/debounce-time.txt

قانون کلی debounceTime:

هرگاه مقداری ارسال شود، یک تایمر با مدت‌زمان ثابت فعال می‌شود. اگر قبل از اتمام این زمان، مقدار جدیدی بیاید، مقدار قبلی دور ریخته شده و تایمر مجدداً ریست می‌شود. تنها زمانی مقدار در خروجی منتشر می‌شود که تایمر بدون دریافت هیچ مقدار جدیدی تا انتها سپری شود.

throttle

منبع: throttle/throttle.txt

اینجا) durationSelector منتشر می‌کند، سپس تا زمانی که (leading edge) مقدار را در لبهٔ ابتدایی throttle .کامل شود، مقادیر جدید را سرکوب می‌کند. پس از آن دروازه دوباره باز است (timer

نکته: تفاوت طول بازهٔ سکوت را می‌بینید؛ چون زمان وابسته به خود مقدار ورودی است 10) * (x، وقتی عدد 2 وارد شد زمان قفل شدن دروازه دو برابر (20) شد و توانست ۳ تا از مقادیر بعدی را نادیده بگیرد.

throttleTime

منبع: throtle-time/throtle-time.txt

اولین مقدار را بلافاصله عبور بده، سپس تا زمان مشخصی (اینجا ۲۵ms) تمام مقادیر ورودی را سرکوب کن.

distinct

منبع: distinct/distinct.txt

قانون کلی distinct:

تنها مقادیری در خروجی منتشر می‌شوند که برای اولین بار در طول عمر کل Observable ارسال شده باشند؛ اگر مقداری حتی یک‌بار در گذشته آمده باشد (چه بلافاصله قبل از آن، چه با فاصله زیاد)، برای همیشه فیلتر و دور ریخته می‌شود.

first

منبع: first/first.txt

تنها اولین مقدار منتشرشده (یا اولین مقداری که شرط داده‌شده را پاس کند) را در خروجی منتشر می‌کند و بلافاصله استریم را Complete (و Unsubscribe) می‌کند.

)مشابه take(1)، با این تفاوت که اگر استریم خالی باشد و بدون ارسال مقدار Complete شود، first() یک خطای EmptyError پرتاب می‌کند(.

شرط هم میشه داد به first:

```
of(1, 2, 3, 4, 5).pipe(
  first(x => x % 2 === 0) // اولین عدد زوج
).subscribe(console.log);
```

last

منبع: ignore-elements/last.txt

صبر می‌کند تا استریم ورودی Complete شود؛ سپس آخرین مقدار منتشرشده (یا آخرین مقداری که شرط داده‌شده را پاس کرده) را منتشر و بلافاصله استریم را Complete می‌کند.

پشتیبانی از شرط (Predicate): مانند first، این اپراتور هم شرط و مقدار پیش‌فرض می‌پذیرد:

```
of(1, 2, 3, 4, 5).pipe(
   last(x => x % 2 === 0) // آخرین عدد زوج
 ).subscribe(console.log);
 // 4 :خروجی بعد از اتمام استریم
```

خطا در صورت خالی بودن: اگر استریم بدون ارسال هیچ مقداری (یا هیچ مقداری که شرط را پاس کند) Complete شود، خطای EmptyError پرتاب می‌کند (مگر اینکه defaultValue داده باشید).

sample

منبع: sample/sample.txt

هرگاه Observable دوم (Notifier) مقداری منتشر کند، آخرین مقدار منتشرشده از استریم اصلی (از زمان نمونه‌برداری قبلی تا الان) را در خروجی می‌دهد؛ اگر در آن بازه مقدار جدیدی نیامده باشد، هیچ خروجی‌ای وجود ندارد.

منظور این است که sample مقادیر قدیمی و تکراری را دوباره چاپ نمی‌کند؛ فقط در صورتی خروجی می‌دهد که دیتای «تازه» آمده باشد.

مثال:

```
import { interval } from 'rxjs';
import { sample, switchMap } from 'rxjs/operators';

// (استریم تغییرات فرم (هر بار کاربر کاراکتری تایپ کند مقدار می‌دهد
const formChanges$ = this.myForm.valueChanges;

// تایمر: هر ۵ ثانیه یک سیگنال می‌فرستد
const saveTimer$ = interval(5000);

formChanges$.pipe(
  sample(saveTimer$), // 👈 نمونه‌برداری با تایمر
  switchMap(formData => this.apiService.autoSave(formData))
).subscribe(() => {
  console.log('فرم با موفقیت در پس‌زمینه ذخیره شد.');
});
```

توضیحش در عکس img_1.png

زمان	اکشن کاربر	وضعیت تایمر ($saveTimer)	رفتار sample
ثانیه ۱ تا ۴	کاربر می‌نویسد: Hello World	منتظر است...	چیزی ارسال نمی‌شود
ثانیه ۵	کاربر متوقف شده	تیک ۵ ثانیه می‌زند!	✅ آخرین مقدار (Hello World) ارسال و ذخیره می‌شود
ثانیه ۶ تا ۹	کاربر دست به کیبورد نمی‌زند (بدون تغییر)	منتظر است...	چیزی ارسال نمی‌شود
ثانیه ۱۰	کاربر هنوز هیچ چیزی تایپ نکرده	تیک ۱۰ ثانیه می‌زند!	❌ هیچ درخواستی به سرور نمی‌رود (چون دیتای جدیدی نیامده)
ثانیه ۱۱ تا ۱۴	کاربر متن را تغییر می‌دهد به Hello RxJS	منتظر است...	چیزی ارسال نمی‌شود
ثانیه ۱۵	-	تیک ۱۵ ثانیه می‌زند!	✅ مقدار جدید (Hello RxJS) ارسال و ذخیره می‌شود

skip

منبع: skip/skip.txt

تعداد n مقدار اولی که از استریم می‌آیند را دور می‌ریزد و پس از آن، تمام مقادیر بعدی را بدون تغییر از خود عبور می‌دهد.

یک کاربرد واقعی (Angular):

فرض کنید یک فرم دارید که در لحظه لود شدن صفحه، مقدار اولیه خودش را (که ممکن است null یا مقدار پیش‌فرض باشد) به عنوان valueChanges منتشر می‌کند، اما شما فقط می‌خواهید به تغییراتی که کاربر واقعاً انجام می‌دهد واکنش نشان دهید:

```
this.myForm.valueChanges.pipe(
  skip(1) // مقدار اولیه (Initial Load) را نادیده بگیر
).subscribe(value => {
  // فقط تغییراتی که کاربر واقعاً تایپ کرده اینجا می‌رسد
  console.log('تغییر واقعی توسط کاربر:', value);
});
```

skipUntil

منبع: skip-until/skip-until.txt

تمام مقادیر استریم اصلی را نادیده می‌گیرد (Skip می‌کند) تا لحظه‌ای که Observable دوم اولین مقدار خود را منتشر کند؛ از آن لحظه به بعد، استریم اصلی بدون مانع عبور داده می‌شود.

```
import { Component, OnInit } from '@angular/core';
import { FormControl, FormGroup } from '@angular/forms';
import { Subject } from 'rxjs';
import { skipUntil } from 'rxjs/operators';

@Component({ ... })
export class EditUserComponent implements OnInit {
  form = new FormGroup({
    username: new FormControl(''),
    email: new FormControl('')
  });

  // تریگری برای اعلام پایان مقداردهی اولیه فرم
  private formInitialized$ = new Subject<void>();

  ngOnInit() {
    // ۱. گوش دادن به تغییرات فرم فقط پس از آماده شدن کامل دیتا
    this.form.valueChanges.pipe(
      skipUntil(this.formInitialized$) // 👈 تا سیگنال نیاید، تمام ایونت‌ها رد می‌شوند
    ).subscribe(() => {
      this.isDirty = true;
      console.log('کاربر واقعاً فرم را ویرایش کرد!');
    });

    // ۲. لود دیتا از API
    this.userService.getUser().subscribe(userData => {
      this.form.patchValue(userData); // این خط valueChanges اما، تولید می‌کند skipUntil را آن فیلتر می‌کند

      // حالا که دیتا ست شد، دروازه را باز کن
      this.formInitialized$.next();
      this.formInitialized$.complete();
    });
  }
}
```

skipWhile

منبع: skip-while/skip-while.txt

تا زمانی که شرط (Predicate) مقدار true برگرداند مقادیر را نادیده می‌گیرد (Skip می‌کند)؛ به محض اینکه شرط برای اولین بار false شد، قفل باز می‌شود و تمام مقادیر بعدی بدون هیچ فیلتری عبور داد

نادیده‌گرفتن مقادیر فرم تا زمانی که فرم به حالت معتبر (Valid) نرسیده است؛ اما پس از اولین بار معتبر شدن، تمام تغییرات بعدی (حتی اگر موقتاً نامعتبر شود) ارسال شوند:

```
this.myForm.statusChanges.pipe(
  skipWhile(status => status === 'INVALID')
).subscribe(status => {
  console.log('اولین بار ولید شد، از الان به بعد مانیتور می‌شود:', status);
});
```

take

منبع: take/take.txt

فقط به تعداد مشخص‌شده از استریم دریافت می‌کند و بلافاصله استریم را Complete (و لغو اشتراک / Unsubscribe) می‌کند.

takeLast

منبع: take-last/take-last.txt

تا زمانی که استریم Complete شود هیچ چیزی منتشر نمی‌کند؛ سپس n مقدار پایانی (آخرین مقادیری که قبل از بسته شدن استریم آمده‌اند) را یک‌جا منتشر می‌کند و سپس بسته می‌شود.

اگر استریم هرگز Complete نشود)مثل interval یا fromEvent)، اپراتور takeLast تا ابد منتظر می‌ماند و هیچ‌وقت هیچ مقداری را منتشر نخواهد کرد! چون نمی‌داند «آخرین مقدار» کجاست.

takeUntil

منبع: take-until/take-until.txt

مقادیر را عبور می‌دهد «تا زمانی که» استریم دوم اولین سیگنال/مقدار را صادر کند؛ با اولین سیگنال استریم دوم، جریان اصلی بلافاصله متوقف و لغو اشتراک (Complete/Unsubscribe) می‌شود.

takeWhile

منبع: take-while/take-while.txt

تا وقتی expression برابر true باشد، مقادیر را منتشر می‌کند؛ به محض اینکه اولین false برسد، جریان را قطع و complete می‌کند.

```
import { Component, inject, OnInit } from '@angular/core';
import { FormBuilder, Validators, ReactiveFormsModule } from '@angular/forms';
import { takeWhile } from 'rxjs';

@Component({
  selector: 'app-user-form',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form">
      <input formControlName="email" placeholder="ایمیل" />
    </form>
  `
})
export class UserFormComponent implements OnInit {
  private fb = inject(FormBuilder);

  form = this.fb.group({
    email: ['', [Validators.required, Validators.email]]
  });

  ngOnInit() {
    this.form.statusChanges.pipe(
      // تا زمانی که فرم INVALID است ادامه بده، به محض اینکه VALID شد استریم را Complete کن
      takeWhile(status => status === 'INVALID')
    ).subscribe(status => {
      console.log('فرم هنوز نامعتبر است:', status);
      this.showValidationHints();
    });
  }

  showValidationHints() {
    // نمایش راهنمای پر کردن فرم
  }
}
```

عملگرهای شرطی و کمکی

defaultIfEmpty

منبع: defaultIfEmpty/defaultIfEmpty.txt

اپراتور defaultIfEmpty در RxJS (معادل دقیق DefaultIfEmpty() در LINQ سی‌شارپ) برای مواقعی است که می‌خواهید مطمئن شوید یک Observable حداقل یک مقدار منتشر می‌کند.

خلاصه عملکرد (در ۳۰ ثانیه):
شرط اجرا: اگر Observable ورودی بدون انتشار هیچ مقداری به پایان برسد (complete شود)، این اپراتور مقدار پیش‌فرضی که به آن داده‌اید را emit کرده و سپس complete می‌شود.
اگر داده وجود داشته باشد: اگر Observable حتی یک مقدار هم emit کند، مقدار پیش‌فرض کاملاً نادیده گرفته شده و همان مقادیر اصلی عبور می‌کنند.

کاربرد رایج:
بعد از فیلترهایی مثل filter() یا first() که ممکن است هیچ آیتمی شرط را پاس نکند و استریم بدون مقدار بسته شود.
درخواست‌های HTTP یا جستجوهایی که نتیجه خالی دارند و می‌خواهید یک State پیش‌فرض (مثل آرایه خالی [] یا شیء پیش‌فرض) در UI دریافت کنید تا کامپوننت دچار خطای null/undefined نشود.

every

منبع: every/every.txt

خلاصه عملکرد (در ۳۰ ثانیه):
شرط اجرا: یک تابع شرطی (predicate) می‌گیرد. اگر تمام مقادیر استریم تا قبل از complete شدن شرط را پاس کنند، مقدار true منتشر می‌شود.
شکست شرط (Short-circuit): به محض اینکه حتی یک مقدار شرط را نقض کند (برگرداندن false)، اپراتور بلافاصله false را منتشر کرده و استریم را متوقف (complete) می‌کند.
استریم خالی: اگر استریم بدون هیچ مقداری complete شود، به طور پیش‌فرض true برمی‌گرداند.

findIndex

منبع: find-index/find-index.txt

ورودی‌ها را به ترتیب (از ایندکس ۰) بررسی می‌کند؛ به محض اینکه اولین مقدار شرط (Predicate) را پاس کند، فقط شماره‌ی ایندکس (موقعیت) آن مقدار را در خروجی منتشر کرده و بلافاصله استریم را Complete (کامل) می‌کند (اگر تا پایان استریم هیچ مقداری شرط را پاس نکند، عدد 1- را emit و complete می‌کند).

sequenceEqual

منبع: sequenceEqual/sequenceEqual.txt

آیا دو استریم دقیقاً مقادیر یکسان با ترتیبی یکسان منتشر می‌کنند یا خیر.

خروجی true: هر دو استریم باید به ترتیب مقادیر برابری بدهند، تعدادشان برابر باشد و هر دو complete شوند.
خروجی false: به محض اینکه یک آیتم در همان ایندکس مغایرت داشته باشد، یا یکی از استریم‌ها زودتر complete شود در حالی که دیگری هنوز آیتم دارد، بلافاصله false منتشر می‌شود.

repeat

منبع: repeat/repeat.txt

اپراتور repeat(count) برای تکرار مجدد یک Observable پس از Complete شدنِ آن استفاده می‌شود.

به محض اینکه استریم مبدا Complete می‌شود، بلافاصله دوباره در آن Subscribe می‌کند و این فرآیند را تکرار می‌کند تا مجموعاً ۳ بار اجرا شود (۱ بار اجرای اصلی + ۲ بار تکرار مجدد).
خروجی نهایی به صورت A, B (بار اول)، A, B (بار دوم)، A, B (بار سوم) منتشر شده و در نهایت استریم اصلی Complete می‌شود (|).

فولدرهایی که فایل نکات ندارند

- audit
- distinc-until-changed
- generator
- iif
- last (نکات last داخل ignore-elements/last.txt است)
- mappers
- others
- single
- switch-map